

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – DAST SIMULATION

|                  |                                                                                               |               |                                                |
|------------------|-----------------------------------------------------------------------------------------------|---------------|------------------------------------------------|
| Course Code:     | CSA5803                                                                                       | Course Title: | DEVSECOPS ENGINEERING AND APPLICATION SECURITY |
| CO Assessed:     | CO3 – Precision (BL1, BL2, BL3)                                                               | Assessment:   | Assessment Tool 2                              |
| Weightage:       | 30% of CIA                                                                                    | Total Marks:  | 100                                            |
| CO3 Statement:   | Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA). |               |                                                |
| Student Name:    | Harish T                                                                                      | Reg. No.:     | 192521373                                      |
| Section / Batch: | B Tech IT 2025                                                                                | Date:         | 13/08/26                                       |

### Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

| Question Type   | Focus Area        | Questions | Marks   |
|-----------------|-------------------|-----------|---------|
| DAST SIMULATION | Application based | Q1        | Q1 –100 |
| GRAND TOTAL:    |                   | 100 Marks |         |

### Code of Conduct

I Harish T (192521373)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Harish T

# DAST SIMULATION – ASSESSMENT TOOL 2

## CSA5803 – DevSecOps Engineering and Application Security

### Q1. DAST and SCA Simulation

#### 1. Aim

To simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA) on a sample web application, identify runtime security weaknesses and risks in third-party dependencies, and recommend suitable remediation measures.

#### 2. Objectives

- Understand the purpose of DAST and SCA in a DevSecOps security process.
- Perform a DAST-style assessment against a running synthetic web application.
- Identify common runtime security weaknesses and classify their severity.
- Analyze project dependencies using SCA concepts and identify vulnerable or outdated components.
- Recommend remediation and verification steps before deployment.

#### 3. Application-Based Scenario

A synthetic web application named SecureShop is considered for the simulation. It contains a login page, product search, product details, and an API endpoint. The application is assumed to run in a controlled local testing environment. DAST evaluates externally observable runtime behavior, while SCA examines the third-party packages declared by the project.

#### 4. DAST Simulation Workflow

Start the application → Identify the application URL and exposed endpoints → Crawl accessible pages → Send controlled security test requests → Observe HTTP responses and application behavior → Record findings → Classify severity → Apply fixes → Re-test the application.

#### 5. Simulated DAST Findings

| ID      | Finding                       | Severity | Risk / Impact                                                  | Recommended Remediation                                 |
|---------|-------------------------------|----------|----------------------------------------------------------------|---------------------------------------------------------|
| DAST-01 | Missing security headers      | Medium   | Reduced browser-side protection against some attacks.          | Configure appropriate HTTP security headers.            |
| DAST-02 | Insufficient input validation | Medium   | Unexpected or malicious input may affect application behavior. | Validate input server-side and use safe query handling. |
| DAST-03 | Verbose error response        | Low      | Detailed errors can reveal                                     | Return generic errors and securely log diagnostics.     |

|         |                                        |      |                                                                            |                                                           |
|---------|----------------------------------------|------|----------------------------------------------------------------------------|-----------------------------------------------------------|
|         |                                        |      | implementation information.                                                |                                                           |
| DAST-04 | Session cookie lacks secure attributes | High | Improperly protected session cookies can increase session-compromise risk. | Use Secure, HttpOnly and appropriate SameSite attributes. |

## 6. SCA Simulation

Software Composition Analysis examines open-source and third-party dependencies used by an application. The dependency manifest is reviewed and components are checked for known vulnerabilities, outdated versions, and unnecessary dependencies. Findings are then prioritized for upgrade, replacement, removal, or further assessment.

## 7. Simulated SCA Findings

| ID     | Dependency                                   | Issue                                                    | Severity | Recommended Action                                                  |
|--------|----------------------------------------------|----------------------------------------------------------|----------|---------------------------------------------------------------------|
| SCA-01 | Example HTTP library – older version         | Known security issue in an outdated version              | High     | Upgrade to a supported patched version and re-scan.                 |
| SCA-02 | Example utility package – outdated version   | Dependency is no longer current                          | Medium   | Update to a maintained release after compatibility testing.         |
| SCA-03 | Example logging package – vulnerable release | Known vulnerability associated with the selected release | High     | Upgrade to a secure release and verify transitive dependencies.     |
| SCA-04 | Unused third-party package                   | Unnecessary dependency increases attack surface          | Low      | Remove the unused dependency and regenerate dependency information. |

## 8. Risk Prioritization

High - severity findings should be addressed first because they can create significant security exposure. Medium - severity findings should be remediated in the normal security - fix cycle, while low - severity findings should be tracked and resolved as part of application hardening. Actual severity should be confirmed using exploitability, exposure, affected component, and business impact.

## 9. Remediation Plan

- Fix identified runtime weaknesses and re-test affected endpoints.
- Apply secure HTTP response headers and secure session-cookie attributes.
- Implement server-side input validation and safe query handling.
- Upgrade vulnerable third-party dependencies after compatibility testing.
- Remove unnecessary dependencies to reduce the application's attack surface.

- Run DAST and SCA again after remediation and document the final status of every finding.

## 10. DAST vs SCA

| Aspect          | DAST                                                 | SCA                                                |
|-----------------|------------------------------------------------------|----------------------------------------------------|
| Primary focus   | Security of the running application                  | Security of third-party/open-source dependencies   |
| Analysis target | Application behavior, endpoints and HTTP responses   | Direct and transitive software components          |
| Typical timing  | When an application is available to test             | During development/build and dependency management |
| Main outcome    | Runtime security findings                            | Dependency vulnerabilities and outdated components |
| Remediation     | Fix application configuration and runtime weaknesses | Upgrade, replace or remove affected dependencies   |

## 11. DevSecOps Integration

DAST and SCA can be incorporated into CI / CD as complementary security controls. SCA can scan dependencies during builds or pull requests and flag vulnerable components before release. DAST can run against a deployed test instance to identify runtime weaknesses. Security gates can be configured for agreed severity levels, followed by remediation and re-testing before production deployment.

## 12. Expected Observations

- DAST identifies weaknesses observable from the behavior of a running application.
- SCA identifies security risks associated with third-party and open-source dependencies.
- The two techniques examine different layers and therefore complement one another.
- Prioritizing high-severity findings improves remediation efficiency.
- Re-testing is necessary to confirm that identified issues have been addressed.

## 13. Result

The DAST and SCA simulation was successfully performed conceptually on a synthetic web application. The DAST assessment identified runtime issues such as missing security headers, insufficient input validation, verbose error responses and insecure session - cookie configuration. The SCA assessment identified outdated or vulnerable third - party dependencies and an unnecessary dependency. Appropriate remediation actions were defined, demonstrating how DAST and SCA support continuous security in a DevSecOps workflow.

## 14. Conclusion

DAST helps detect security weaknesses in an application while it is running, whereas SCA focuses on security risks introduced through software dependencies. Using both techniques provides broader application security coverage. Integrating them into the DevSecOps lifecycle enables earlier detection, systematic remediation and security verification before software reaches production.